Project 15 Literature Review

Project: Build a Custom Harness and Beat an Established One on Terminal-Bench

Subject: 36127 Innovation Lab: Capstone Project

Prepared for: Six-member Project 15 team

Date: 30 July 2026

Project context

Project 15 asks the team to build a custom agent harness and test whether it can outperform an established harness on Terminal-Bench while holding the underlying language model constant. The project brief narrows the intended scope:

- use Terminal-Bench 2.1 and its 89 terminal tasks;
- compare against two established harnesses;
- hold the model constant so that performance differences can be attributed more credibly to harness design;
- develop on a fixed subset of 20 tasks;
- change one design lever at a time;
- run the full benchmark only after the custom harness design is frozen;
- record accuracy, tokens, cost, and design changes; and
- submit the resulting harness to the public leaderboard if resources and the final protocol allow.

This framing makes the Capstone a controlled experimental software-engineering project. The main deliverable is not merely an agent that completes terminal tasks. The team must produce defensible evidence about which harness decisions help, which do not, and under what cost and reliability conditions.

The literature selected for this review addresses five complementary questions:

1. What does Terminal-Bench measure, and what common agent failures does it expose?
2. How can an interface designed for an LLM improve performance without changing model weights?
3. What components appear in a general software-agent platform?
4. When can a simple, constrained workflow outperform a complex autonomous agent?
5. How should an agent system be structured for reproducibility, recovery, and reliable experimentation?

Executive summary

The five papers support a consistent conclusion: the underlying model is only one part of an agentic system. Performance also depends on the interface between the model and its environment, the information retained in context, the feedback supplied after actions, the checks performed before completion, and the mechanisms used to recover from errors.

Terminal-Bench provides 89 difficult, outcome-verified terminal tasks. Its failure analysis highlights behaviours directly relevant to harness design: failing to follow specifications, repeating ineffective steps, stopping prematurely, hallucinating results, failing to verify core requirements, and claiming success despite contradictory

evidence. Terminal-Bench 2.1 subsequently corrected issues in 28 of the original 89 tasks, demonstrating that benchmark configuration and verifier quality can materially change measured performance.

SWE-agent offers the strongest direct evidence that interface design matters. It introduces the agent-computer interface, or ACI, and shows that simple actions, compact operations, concise feedback, and guardrails can improve an agent while keeping the model fixed. Its ablations suggest that summarised search, bounded file views, compact editing operations, linting, and reduced observation history can be more effective than exposing an unstructured shell alone.

OpenHands demonstrates the breadth of a general software-agent platform: agent logic, tools, sandboxed execution, event history, state management, model abstraction, applications, and benchmark adapters. It is valuable as an established baseline and architectural reference, but its breadth is also a warning against reproducing a production platform within a one-semester Capstone.

Agentless shows that complex autonomy is not always necessary. A staged workflow of localisation, repair, and validation can achieve strong results at relatively low cost. Candidate selection and reproduction testing are central to its performance. For Project 15, this supports beginning with a minimal, interpretable loop and adding autonomy only when controlled experiments demonstrate benefit.

The OpenHands SDK paper contributes production-oriented design lessons: separate agent logic from applications, keep configuration immutable, maintain a single authoritative state, store actions and observations as events, and isolate infrastructure failures from model failures. These practices are particularly important for a Capstone in which every result must be reproducible and explained.

The recommended initial harness is therefore a minimal external Harbor agent with:

- a fixed prompt and explicit plan-execute-verify workflow;
- one terminal execution interface initially;
- bounded context with a compact state summary;
- structured action and observation records;
- completion gated by observed verification;
- one error-aware repair opportunity;
- immutable configuration for every run; and
- metrics for accuracy, tokens, cost, runtime, retries, and failure type.

The first experiments should test prompt structure, verification gating, error-aware repair, context management, and tool granularity in that order. Multi-agent orchestration, fine-tuning, unconstrained web access, and elaborate memory systems should remain outside the initial scope.

Key terminology

Term	Working definition for Project 15
Model	The underlying large language model that generates reasoning, commands, and responses. It must remain fixed during a controlled harness comparison.
Agent	The model plus the policy or loop that decides how to act in an environment.
Harness	The surrounding system that connects the model to tasks, tools, execution, context, logging, retries, and evaluation.
Agent-computer interface	The actions available to an agent and the observations returned by the environment.
Harbor	The evaluation framework used to run agents in containerised task environments and collect trial results.
Task	An instruction, container environment, resource policy, and verifier that define one benchmark problem.
Verifier	Automated tests or checks that judge the final environment state.
Trajectory	The ordered record of prompts, actions, observations, tool results, state changes, and completion.
Baseline	An established harness run under the same model and benchmark conditions as the custom harness.

Term	Working definition for Project 15
Development subset	The fixed 20-task subset used to design and compare harness changes.
Final evaluation	The frozen custom harness run on the full 89-task benchmark under an agreed protocol.
Ablation	A controlled comparison in which one component is removed or changed to estimate its contribution.
Reward hacking	Achieving a verifier score without satisfying the intended task outcome.

Paper 1: Terminal-Bench

Research problem

The Terminal-Bench paper argues that many existing agent benchmarks are either too artificial or insufficiently difficult to measure frontier agents. Terminal work is a valuable test domain because it includes realistic activities from software engineering, machine learning, cybersecurity, scientific computing, system administration, and research reproduction.

Proposed benchmark

Terminal-Bench 2.0 contains 89 manually curated tasks. Each task includes:

- an English instruction;
- a containerised environment;
- tests that examine the final container state;
- a human-written reference solution; and
- time and resource constraints.

The benchmark is outcome-driven. It generally evaluates the final state rather than requiring the agent to follow one prescribed command sequence. This permits multiple valid solutions while retaining objective verification.

Reported finding: In the original evaluation, frontier model-agent combinations solved fewer than 65% of tasks, while substantially smaller models performed much worse. The benchmark therefore retained enough difficulty to expose differences in agent and harness behaviour.

Failure analysis

The paper provides a useful taxonomy of behaviours that prevent completion:

- Specification failure: the agent violates a required format, method, path, or other task constraint.
- Step repetition: the agent repeats an unsuccessful action without learning from the result.
- Premature termination: the agent stops before producing or validating the required outcome.
- Hallucination or guessing: the agent substitutes an unsupported answer when required information is unavailable.
- No or irrelevant verification: the agent does not observe evidence about core functional requirements.
- Weak verification: checks exist but do not cover the properties required for genuine correctness.
- Reasoning-action mismatch: the agent claims or plans one thing while its commands and artifacts show another.
- Data fabrication or evaluator manipulation: the agent creates or alters evidence that should have been measured or derived legitimately.

The distinction between no verification and weak verification is important. An agent that never runs a check has a different failure from an agent that runs a superficial check and mistakenly treats it as proof.

Terminal-Bench 2.1

Project 15 specifies Terminal-Bench 2.1 rather than the original 2.0 release described in the paper. Version 2.1 keeps the 89-task benchmark but fixes 28 tasks affected by changing external dependencies, unsuitable resource limits, verifier weaknesses, and robustness problems. Official comparisons show that these corrections changed scores materially for several model-harness combinations.

Reported finding: Benchmark corrections can alter measured accuracy by multiple percentage points even when the agent and model are unchanged.

Project implication: The team must pin the exact dataset, Harbor version, harness versions, model, reasoning effort, resource policy, network policy, timeouts, and trial count. Otherwise, a configuration change could be mistaken for an improvement in the custom harness.

Evaluation implications

Accuracy is the primary outcome, but a credible study should also report:

- per-task success;
- uncertainty or variation across repeated trials;
- token consumption;
- cost;
- runtime;
- error and timeout rates;
- task categories;
- failure types; and
- cases in which a verifier may not represent intended correctness.

Limitations

No benchmark fully represents real professional work. Terminal-Bench tasks are containerised and time-bounded, and performance may depend on model training exposure, infrastructure, network availability, and verifier coverage. Public benchmarks also face contamination and overfitting risks.

Candidate experiment

Compare a baseline completion policy against a verification-gated policy:

- Control: the agent may stop when it declares completion.
- Treatment: the harness requires a relevant observed check; failed checks are returned to the model for one repair attempt.

Measure full-task accuracy, premature completion, verification coverage, tokens, cost, and runtime.

Paper 2: SWE-agent

Research problem

SWE-agent asks whether interfaces designed for human developers are also appropriate for language-model agents. Humans use rich editors and can ignore irrelevant information, interpret extensive documentation, and recover flexibly from mistakes. LLMs have different context, attention, and action limitations.

Agent-computer interface

The paper defines an ACI as both:

- the actions that the model can take; and

- the representation of environment state and feedback supplied after those actions.

This definition is broader than a tool list. It includes command documentation, output formatting, file windows, history processing, error messages, and workflow guardrails.

Four design principles

1. Actions should be simple. Tool names, parameters, and instructions should be easy for the model to interpret.
2. Actions should be compact and efficient. An operation should make meaningful progress without requiring many fragile intermediate turns.
3. Feedback should be informative but concise. The agent needs evidence about what changed, but unnecessary output consumes context and distracts from the task.
4. Guardrails should prevent error propagation. Syntax checks and constrained editing help the agent detect and correct mistakes earlier.

Architecture

SWE-agent uses a repeated reasoning-action-observation loop with specialised commands for:

- file and symbol search;
- bounded file viewing;
- editing;
- test or program execution; and
- context/history management.

It still allows ordinary shell commands when required, but common software-engineering operations receive model-friendly interfaces.

Evaluation and ablations

The paper compares SWE-agent with non-interactive retrieval approaches and a shell-only interactive baseline while keeping the underlying model fixed for relevant comparisons.

> Reported finding: On the SWE-bench Lite ablation subset, the complete interface achieved a large improvement over the shell-only baseline. The study also found that individual interface choices materially changed resolution rates.

Specific ablation findings include:

- summarised search results outperformed an iterative interface that encouraged exhaustive inspection;
- a bounded file window outperformed displaying a whole file;
- retaining the last few detailed observations outperformed retaining the complete history;
- compact multi-line editing was important;
- automatic linting improved editing reliability; and
- additional tools could reduce performance when they encouraged inefficient behaviour.

Cost and context

The study uses a per-instance cost budget. This makes agent efficiency part of the design problem: an interface that causes excessive browsing, output, or repeated actions can exhaust the budget before the task is solved.

> Project implication: Measure the number of turns, input tokens, output tokens, and repeated actions, not only pass/fail. A harness improvement that gains little accuracy while doubling cost may not be defensible.

Limitations

The paper focuses on software-engineering benchmarks rather than the full domain diversity of Terminal-Bench. Results were obtained using particular models, prompts, tools, and cost limits. An ACI that helps one model may not help another equally.

Candidate experiment

Compare three observation strategies:

1. full raw terminal output and full history;
2. raw recent output with a sliding history window; and
3. bounded output plus a persistent task-state summary and recent actions.

Measure accuracy, token use, context-limit errors, repeated actions, and recovery from failures.

Paper 3: OpenHands

Research problem

OpenHands presents an open platform for building generalist software-development agents. It addresses the need to combine agent policies, model access, code execution, terminal interaction, browsing, sandboxing, state, user interfaces, and evaluation within one extensible system.

Platform architecture

The platform demonstrates several components that also appear in a Terminal-Bench harness:

- a model abstraction;
- agent logic;
- a library of actions and observations;
- a runtime that executes actions;
- an isolated workspace;
- conversation and event history;
- controller logic;
- application interfaces; and
- benchmark adapters.

Agents can write code, interact with a command line, use browser capabilities, and operate within sandboxed environments. The platform also supports research comparison across agents and benchmarks.

Relevance as an established harness

OpenHands is a plausible baseline for Project 15 because Harbor currently includes an integration for it. Its maturity makes it a meaningful comparison, but fair use requires pinning its version and configuring it to use the same model and equivalent benchmark constraints.

Reported finding: OpenHands demonstrates that generalist software agents can be implemented as modular policies operating through a common action/observation system and a controlled runtime.

Project implication: Reuse architectural boundaries, not the entire platform. The custom Capstone harness should expose only the functionality needed to test selected design hypotheses.

Risks of excessive scope

OpenHands includes capabilities that are not necessary for the minimum Project 15 deliverable:

- graphical applications;

- broad browsing;
- multiple application front ends;
- complex multi-agent coordination;
- production deployment integrations; and
- extensive general-purpose tool ecosystems.

Attempting to reproduce these features would consume the semester without answering the central research question.

Limitations

The original OpenHands paper evaluates a broad platform across multiple tasks and agents. It is not a controlled study of every individual harness component. Some performance is inseparable from the models, prompts, tools, and benchmark adapters used at evaluation time.

Candidate experiment

Use OpenHands as one established baseline and compare its trajectory-level behaviour with the custom harness on the same development tasks:

- number and type of actions;
- proportion of actions that change task state;
- repeated or failed commands;
- verification behaviour;
- turns before completion; and
- cost per successful task.

The analysis may reveal why the custom harness wins or loses rather than reporting accuracy alone.

Paper 4: Agentless

Research problem

Agentless challenges the assumption that increasingly complex autonomous agents are always the best approach to software-engineering problems. Autonomous agents can be expensive, difficult to reproduce, prone to wandering, and hard to analyse.

Three-stage workflow

Agentless uses a constrained pipeline:

1. Localisation: identify relevant files, classes, functions, and edit locations.
2. Repair: generate multiple candidate patches for selected locations.
3. Patch validation: use regression and generated reproduction tests to filter and rank candidates.

The model performs focused generation within these stages rather than freely selecting tools and future actions at every turn.

Main findings

Reported finding: On SWE-bench Lite, Agentless reported 32% resolution with relatively low cost, outperforming the open-source agent-based approaches compared in that study.

Its ablations show that:

- hierarchical localisation reduces context while retaining relevant code;
- multiple location and patch samples improve the chance of finding a correct repair;

- candidate selection is a major performance bottleneck;
- regression testing improves selection;
- generated reproduction tests provide a further substantial improvement; and
- more samples eventually provide diminishing practical value.

The authors also identify problematic benchmark tasks with missing, misleading, or leaked solution information and propose a filtered benchmark.

Relevance to Project 15

Agentless provides two important lessons:

1. A structured, narrow workflow is a credible baseline, not merely a simplified prototype.
2. Verification and selection may contribute more than unconstrained planning.

The Capstone harness can incorporate these lessons without becoming entirely non-agentic. A minimal loop can constrain behaviour:

```
Inspect -> Plan -> Execute -> Verify -> Repair once -> Finish
```

Limitations

Agentless evaluates issue resolution in code repositories rather than diverse terminal tasks. Its localisation-repair-validation pipeline may not transfer directly to tasks such as system configuration, security, data processing, or research reproduction.

Candidate experiment

Compare:

- an open-ended ReAct-style loop; and
- a staged inspect-plan-execute-verify-repair policy.

Hold tools and model constant. Evaluate success, turns, cost, repeated actions, and failure-category distribution.

Paper 5: OpenHands Software Agent SDK

Research problem

The OpenHands SDK paper describes lessons from redesigning a monolithic research platform into a composable foundation for reliable software agents. As an agent system grows, tightly coupled runtime, application, state, and tool logic makes experiments difficult to reproduce and failures difficult to diagnose.

Four architectural principles

1. Optional isolation: support local execution where appropriate while retaining sandboxed execution for safety and resource control.
2. Stateless components and one authoritative mutable state: immutable constructed components reduce configuration drift, while one state record enables recovery.
3. Strict separation of concerns: decouple the core agent from user interfaces and applications.
4. Composable packages and typed components: allow tools, workspaces, models, and applications to change through clear interfaces.

Event-sourced state

Rather than treating logs as secondary output, the SDK records state transitions as events. This supports:

- deterministic replay;
- debugging;

- recovery after interruption;
- auditability;
- trajectory analysis; and
- comparison between system versions.

> Reported finding: The paper reports a 61% reduction in system-attributable failures after the redesigned architecture, with low state-persistence and recovery overhead in its production comparison.

Failure separation

The paper distinguishes failures caused by:

- infrastructure and orchestration;
- internal SDK logic; and
- external model providers.

Project 15 should add benchmark/task failure as a separate category. Without this separation, a timeout caused by Docker or an API error may be incorrectly counted as a reasoning failure.

Relevance to Project 15

The custom harness should separate:

- immutable experiment configuration;
- agent policy;
- prompt templates;
- terminal interface;
- context manager;
- verification policy;
- trajectory writer; and
- Harbor adapter.

This decomposition allows the team to change one experimental lever without changing unrelated behaviour.

Limitations

The SDK paper focuses on production reliability and a broad software-agent ecosystem. Project 15 does not require a production service, multi-tenant access, graphical clients, or remote deployment platform. Security analysis remains imperfect, and the reported production results do not directly establish Terminal-Bench gains.

Candidate experiment

Test whether an event-based compact state improves recovery:

- control uses ordinary chat history;
- treatment stores explicit events and produces a compact state summary after a token threshold.

Evaluate context errors, successful continuation after long outputs, tokens, and accuracy.

Cross-paper synthesis

Source	Main contribution	Harness lever	Evidence	Limitation	Project use
Terminal-Bench	Difficult outcome-verified terminal benchmark and failure taxonomy	Verification, stopping, failure analysis	89 tasks expose substantial unsolved performance and recurrent behavioural failures	Public benchmark and verifier limitations	Primary benchmark and failure-classification framework

Source	Main contribution	Harness lever	Evidence	Limitation	Project use
SWE-agent	Agent-computer interfaces tailored to model limitations	Tools, feedback, context, guardrails	Interface ablations materially affect fixed-model performance	Primarily software repository tasks	Highest-priority source for tool and context experiments
OpenHands	General, open software-agent platform	Runtime separation, actions/observations, sandboxing	Demonstrates broad agent-platform integration	Too broad to reproduce within the Capstone	Established baseline and architecture reference
Agentless	Simple staged alternative to autonomous agents	Workflow constraints, candidate validation	Strong reported accuracy/cost and verification ablations	Domain-specific pipeline	Evidence for a minimal staged harness and verification focus
OpenHands SDK	Modular, event-sourced production architecture	State, logs, immutability, recovery	Reported reliability improvement after redesign	Production focus exceeds project scope	Reproducible component boundaries and failure separation

Theme 1: Harness design is experimentally meaningful

SWE-agent shows that interface decisions can improve performance with the model fixed. Terminal-Bench demonstrates that agents fail for behavioural reasons that a harness can influence. This directly supports Project 15's premise.

Theme 2: More autonomy and more tools are not always better

SWE-agent reports inefficient behaviour with some search interfaces, and Agentless demonstrates the strength of a constrained pipeline. The team should justify every added capability with an observable failure it is intended to reduce.

Theme 3: Verification is a control mechanism

Terminal-Bench identifies missing and weak verification as failure modes. Agentless shows that tests improve candidate selection. Verification should therefore be part of the agent policy rather than a final reporting step.

Theme 4: Context is a limited experimental resource

Full terminal output and history are not free. They increase token consumption, retain obsolete state, and may reduce the number of useful turns. SWE-agent's bounded observations and the OpenHands SDK's explicit state management support testing compact context.

Theme 5: Reproducibility requires architecture

Immutable configuration, structured trajectories, version pinning, and failure separation are not merely administrative practices. They are necessary to attribute a score difference to a harness change.

Implications for our custom harness

Recommended minimum architecture

```
Harbor task instruction
|
Immutable run configuration
|
Prompt and staged agent policy
|
Terminal action parser
|
Harbor BaseEnvironment execution
|
Bounded observation processor
|
Event/trajectory record
|
Verification gate
|
Limited error-aware repair
|
Completion or budget exhaustion
```

Recommended component boundaries

Component	Responsibility	Must not control
Configuration	Dataset, model, limits, prompt version, lever settings	Runtime decisions
Agent policy	Decide the next phase and request the next action	Execute commands directly
Prompt strategy	Render task, state, tool documentation, and instructions	Store mutable trajectory state
Tool interface	Parse and validate requested terminal actions	Decide whether the task is complete
Environment adapter	Execute through Harbor and return results	Rewrite model output
Context manager	Retain current state and bounded recent evidence	Change benchmark configuration
Verification policy	Require evidence and trigger bounded repair	Alter the official verifier
Trajectory logger	Persist events and metrics	Influence agent behaviour

Deliberate exclusions from the first version

- multiple cooperating agents;
- model fine-tuning;
- reinforcement learning;
- unrestricted internet search;
- a graphical user interface;
- dynamic model routing;
- a general plugin marketplace;
- persistent cross-task memory; and
- task-specific hard-coded solutions.

These features can be revisited only if the minimal harness is stable and an observed failure cannot be addressed by a smaller change.

Experimental variables

Experiment 1: Prompt structure

- Control: direct autonomous completion instruction.
- Treatment: explicit inspect-plan-execute-verify workflow.
- Hypothesis: staged instructions reduce specification failures and premature completion at small token cost.

Experiment 2: Verification gating

- Control: accept model-declared completion.
- Treatment: require an observed relevant check before completion.
- Hypothesis: verification gating improves accuracy by reducing unverified completion.

Experiment 3: Error-aware repair

- Control: no special action after a failed check.
- Treatment: return the failure evidence and permit one focused repair.
- Hypothesis: one repair attempt recovers a useful proportion of failures without causing uncontrolled loops.

Experiment 4: Context management

- Control: full history and raw output.
- Treatment A: sliding window of recent observations.
- Treatment B: persistent state summary plus recent actions and bounded output.
- Hypothesis: compact state reduces tokens and repetition while maintaining or improving accuracy.

Experiment 5: Tool granularity

- Control: one general terminal interface.
- Treatment: separate compact interfaces for search, file inspection, editing, and execution.
- Hypothesis: structured tools reduce inefficient commands and editing errors, but excessive tool choice may offset the gain.

One-lever rule

Each comparison must keep all other configuration values constant. If a necessary bug fix affects both control and treatment, rerun both conditions or exclude affected results with a documented rationale.

Evaluation framework

Primary outcome

- Task accuracy: proportion of tasks passing the official verifier.

Secondary outcomes

- total and per-task input tokens;
- total and per-task output tokens;
- monetary cost;
- wall-clock runtime;
- number of model turns;
- number of terminal actions;
- retry count;
- timeout rate;
- infrastructure error rate;

- verification coverage;
- repeated-action rate; and
- failure-category distribution.

Development protocol

1. Agree with the mentor on the exact 20-task development subset.
2. Freeze and record the subset before optimising the harness.
3. Run a three-to-five-task infrastructure smoke set.
4. Reproduce two established harness baselines with the same model.
5. Establish the minimal custom harness baseline.
6. Change one design lever at a time.
7. Repeat borderline comparisons where budget permits.
8. Select the final design using a pre-agreed rule rather than subjective preference.
9. Freeze code, configuration, and dependencies.
10. Run the full 89 tasks only after the freeze.

Statistical analysis

Because task outcomes are paired binary results, compare harnesses task by task. Report:

- accuracy and raw task counts;
- paired wins, losses, and ties;
- confidence intervals;
- McNemar's test when repeated protocol and sample size make it appropriate;
- sensitivity to infrastructure failures;
- performance by task category; and
- cost per successful task.

Statistical significance should not replace practical interpretation. A small accuracy gain with a large cost increase may not be a useful improvement.

Leaderboard versus academic evaluation

The official Terminal-Bench 2.1 submission process requires at least five trials per task for a public leaderboard entry. This implies at least 445 custom-harness runs before baselines and development experiments. The team must obtain written confirmation of model access, API funding, infrastructure, and submission expectations before treating leaderboard submission as guaranteed.

Failure taxonomy

Category	Operational definition	Example harness response
Specification	Required method, path, format, or constraint violated	Reinforce constraints and check artifacts before completion
Repetition	Same ineffective action pattern recurs without new evidence	Detect action similarity and request a revised diagnosis
Premature completion	Agent stops without satisfying or checking core requirements	Apply verification gate
Hallucination/guessing	Unsupported result substitutes for missing evidence	Require evidence provenance and prohibit unsupported completion

Category	Operational definition	Example harness response
No verification	No relevant core check is observed	Request a targeted check
Weak verification	Check does not cover required properties	Provide verification checklist or official test command where allowed
Reasoning-action mismatch	Claims contradict commands, errors, or artifacts	Return contradictory evidence to the model
Context failure	Important state lost or context limit reached	Use compact event-derived state
Tool/parser failure	Model action cannot be interpreted or executed	Return concise schema error and allow corrected action
Infrastructure	Docker, Harbor, storage, or orchestration fails	Exclude or rerun under the documented infrastructure policy
Provider	Authentication, rate limit, or model API fails	Record separately and rerun only under the agreed policy
Benchmark/verifier	Task environment or verifier is defective or unstable	Document, isolate, and consult mentor rather than patching silently
Budget exhaustion	Token, turn, time, or cost limit is reached	Save final state and classify without hidden extension

Reproducibility, cost, and validity

Immutable run record

Every trial should store:

- experiment and trial ID;
- timestamp;
- Git commit;
- Terminal-Bench dataset and revision;
- Harbor version and commit where available;
- baseline/custom harness name and version;
- model identifier;
- provider;
- reasoning effort;
- temperature and sampling controls;
- prompt version and hash;
- tool configuration;
- context policy;
- retry policy;
- timeout and resource settings;
- network policy;
- task identifier;
- final reward or pass/fail;
- tokens, cost, and runtime;
- error details;
- failure classification; and
- trajectory/artifact path.

Cost controls

- use oracle runs only to validate infrastructure, not to tune the agent;
- use three to five tasks for pipeline debugging;
- avoid rerunning established baselines when saved results remain protocol-compatible;
- enforce per-trial token, time, and cost limits;
- preserve every usable trajectory;
- perform full runs only after a design freeze;
- estimate the remaining experiment budget weekly; and
- distinguish cached, estimated, and provider-reported cost.

Internal validity risks

- changing more than one harness feature;
- using different model settings between harnesses;
- silently rerunning only failed conditions;
- selecting development tasks after observing results;
- infrastructure changes between treatments;
- task-specific prompt tuning; and
- excluding failures inconsistently.

External validity risks

- results may not transfer to another model;
- the 20-task subset may not represent all 89 tasks;
- Terminal-Bench may not represent interactive professional work fully;
- public benchmark exposure may affect model familiarity; and
- a Terminal-Bench gain may not transfer to SWE-bench or other agent benchmarks.

Construct validity risks

- official tests may omit important task properties;
- pass/fail may hide partial progress;
- token cost may be reported differently across providers;
- runtime may be dominated by infrastructure rather than harness policy; and
- leaderboard ranking may encourage optimisation that does not generalise.

Research questions and hypotheses

Primary research question

How do selected harness-design choices affect task accuracy, cost, and reliability on Terminal-Bench 2.1 when the underlying model is held constant?

Supporting questions

1. Does an explicit inspect-plan-execute-verify workflow outperform a direct autonomous prompt?
2. Does verification-gated completion reduce premature or unsupported success claims?
3. Does one error-aware repair attempt improve accuracy enough to justify its additional tokens and runtime?

4. Does compact state management reduce repetition and context failures without removing necessary evidence?
5. Do structured terminal tools outperform a general shell interface?
6. Which failure categories are most strongly affected by each design change?
7. Can a minimal custom harness outperform at least one established harness on the fixed development subset and full evaluation?

Pre-registered working hypotheses

- H1: Staged prompting will reduce specification and premature-completion failures.
- H2: Verification gating will improve pass rate more than it increases cost.
- H3: A single evidence-guided repair will recover failures efficiently; unrestricted retries will not be necessary.
- H4: State summary plus recent evidence will use fewer tokens than full history with no loss of accuracy.
- H5: A small set of structured tools will reduce editing and navigation failures, but too many tools will increase selection errors.

Six-member reading allocation

Member	Primary reading	Required output	Cross-review
Member 1	Terminal-Bench paper and 2.1 release notes	Benchmark structure, failure taxonomy, version-control checklist	Review Member 4's benchmark-validity notes
Member 2	SWE-agent	ACI design principles, ablation table, candidate interface experiments	Review Member 5's architecture notes
Member 3	OpenHands platform paper	Baseline architecture, Harbor comparison questions, scope exclusions	Review Member 6's reproducibility notes
Member 4	Agentless	Minimal workflow, validation strategy, simplicity/cost argument	Review Member 1's failure taxonomy
Member 5	OpenHands SDK	State, event logging, modularity, failure separation	Review Member 2's experiment proposals
Member 6	Cross-paper synthesis and official Harbor docs	Shared terminology, evidence table, mentor questions, reference audit	Review Member 3's baseline analysis

Each member should prepare:

- a one-page summary;
- three reported findings;
- two limitations;
- two implications for Project 15;
- one candidate experiment;
- one question for the mentor; and
- exact page or section references.

Pair assignments:

- Pair A - Members 1 and 2: Harbor environment and established baselines.
- Pair B - Members 3 and 4: custom harness and agent behaviour.
- Pair C - Members 5 and 6: evaluation, analysis, and documentation.

Pairs should rotate secondary duties at the Week 5, Week 7, and Week 9 milestones so every member gains technical, analytical, and communication experience.

Recommended reading order

1. This literature review - shared vocabulary and project direction.
2. Terminal-Bench paper - benchmark contract, evaluation, and failures.
3. Terminal-Bench 2.1 release and run documentation - current dataset and commands.
4. SWE-agent - highest-priority evidence for interface experiments.
5. Agentless - evidence for minimal staged workflows and validation.
6. OpenHands platform paper - established harness and platform reference.
7. OpenHands SDK paper - reproducible and reliable architecture.
8. Harbor agent documentation - concrete custom-agent integration.

The whole team should read the Terminal-Bench abstract, task formulation, evaluation, and failure analysis. Assigned members then lead the deeper reading and teach the group.

Immediate mentor questions

1. Which exact model and reasoning setting must remain fixed?
2. Which two established harnesses should be used?
3. Will the client or university provide API credits or compute?
4. Is the 20-task development subset provided, or must the team select and freeze it?
5. How many repeated trials are expected during development and final evaluation?
6. Does “beat an established harness” refer to the development subset, one full run, repeated full runs, or an accepted leaderboard submission?
7. What infrastructure failures may be rerun, and how must reruns be reported?
8. Are model/provider substitutions permitted if the preferred configuration is unaffordable?
9. Which deliverables are expected at the Week 5, Week 7, and Week 9 client meetings?
10. What repository, publication, and leaderboard attribution rules apply?

References

1. Merrill, M. A., et al. (2026). Terminal-Bench: Benchmarking Agents on Hard, Realistic Tasks in Command Line Interfaces. https://arxiv.org/abs/2601.11868
2. Terminal-Bench Team. (2026). Terminal-Bench 2.1. https://www.tbench.ai/news/terminal-bench-2-1
3. Terminal-Bench Team. (2026). How to run Terminal-Bench 2.1. https://www.tbench.ai/docs/run-terminal-bench-2-1
4. Yang, J., et al. (2024). SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. https://arxiv.org/abs/2405.15793
5. Wang, X., et al. (2024). OpenHands: An Open Platform for AI Software Developers as Generalist Agents. https://arxiv.org/abs/2407.16741
6. Xia, C. S., et al. (2024). Agentless: Demystifying LLM-based Software Engineering Agents. https://arxiv.org/abs/2407.01489
7. Wang, X., et al. (2026). The OpenHands Software Agent SDK: A Composable and Extensible Foundation for Production Agents. https://arxiv.org/abs/2511.03690
8. Harbor Framework. (2026). Agents: Using popular agents and integrating your own. https://www.harborframework.com/docs/agents
9. Harbor Framework. (2026). Terminus-2 reference agent. https://www.harborframework.com/docs/agents/terminus-2

10 Terminal-Bench Team. (2026). Terminal-Bench 2.1 leaderboard. https://www.tbench.ai/leaderboard/terminal-bench/2.1